RMIT Classification: Trusted

RMIT University

Bachelor of Information Technology - BP162VI23

OENG118X/COSC250X Engineering & IT Capstone Project

Capstone Project Final Report

Konverter: AI Document Intelligence for Legal Publishing

Student team:	4 Bits
Student name & ID:	Aung Su Pyae s4087009
	Linn Htin Nyo s4037889
	Zyle Estacion s4064846
	Ye Thu Aung s4066945
Academic supervisor:	Nhat Quang Tran
Company:	Komosion Pty Ltd
Industry supervisor:	Truc Pham

Contents

1 Executive summary 5
2 Project background & problem statement 6
3 Literature review & market research 8
4 Solution design 13
5 Result analyses & discussions 21
6 Sumary and reflection 24
7 References 25
8 Appendices 26
9 Meeting journals 27

List of Figures

Figure 2-1: Example of figure and caption presentation 6
Figure 4-1: It is important to discuss how the successful design and realization of your project can be measured 13
Figure 5-1: A Gantt chart with brief discussion on your project timeline will work well 21

List of Tables

Table 3-1: Example of table and caption presentation 8

1 Executive summary

It is a summary of your project, providing information about the context/background, the problem that needs to be solved, the solution that will be developed at the end of the project, and its significance/contributions to practice and theory.

Recommended length: 1 page

2 Project background & problem statement

2.1 Project background

Konverter is a document intelligence system developed by the 4 Bits capstone team in collaboration with Komosion Pty Ltd. Komosion's client, the Victorian Law Reform Commission (VLRC), provides the primary application context for the project. The development therefore focuses on transforming VLRC's legal reports into web publications and structured data that can support further digital services.

VLRC reports contain chapters, headings, recommendations, tables, quotations, footnotes and appendices. These elements help readers understand how the information is organized. They form the basis of the project's work on document conversion and online publication. The main users of Konverter are the people from VLRC who are responsible for preparing, reviewing and publishing these reports. Their tasks help define the system's workflow, from uploading a source document to approving its web version.

The project covers the conversion of PDFs into web pages and structured data. Its scope includes document ingestion and processing, human review and publication through WordPress. The system produces accessible HTML for web publication, Schema.org JSON-LD for publication metadata, and structured JSON for downstream applications such as the document-specific AI chatbot. These outputs support both the presentation of reports to readers and their use by other applications.

Throughout two semesters or 6 months, the project developed from investigating extraction and document classification into an integrated review-to-publication system. Early work focused on understanding the documents and identifying suitable processing methods. Later development connected extraction with review tools, output generation, WordPress publishing and AI chatbot. Its immediate scope remains focused on VLRC reports and Komosion's publishing requirements. However, the project also addresses a wider digital transformation issue: how to make information stored in existing PDFs easier for people to access and for software to reuse.

2.2 Problem statement

Preparing a legal PDF for web publication involves more than copying its text. A PDF can preserve how a report looks without clearly identifying how its content is organised. Headings, tables, quotations and footnotes may be recognisable to a reader but difficult for software to interpret correctly. The conversion process must identify these elements and preserve their order and relationships.

Errors in this process can also affect both navigation and understanding. An incorrect heading level can place a section in the wrong part of the report. A misplaced footnote can separate an explanation from the text it supports. A poorly reconstructed table can make information difficult to read. Staff therefore need to compare the converted content with the original PDF, correct its structure and formatting, and check publication details before publishing. Repeating these tasks across long reports creates a substantial workload and takes around 4 hours to complete one PDF.

Many organizations continue to rely heavily on PDF as a primary format and a large amount of information is locked in those PDFs. However, storing information digitally does not automatically make it easy for software to reuse. Auer et al. explain that converting PDFs into machine-readable formats remains difficult because document layouts vary and important structural information may be missing [1]. Thus, software may extract the words without correctly preserving reading order, table relationships or section boundaries.

This creates a challenge when organizations want to use existing documents in search tools, chatbots and other AI applications. The Docling research identifies structured document conversion as an important step in supporting these uses, including retrieval-augmented generation, where an AI system retrieves relevant information from documents to help answer questions [1]. The challenge is therefore to recover both the content and its structure so that information stored in PDFs can be reused reliably.

Moreover, research shows that document accessibility is also a wider problem. Kumar and Wang examined approximately 20,000 scholarly PDFs published between 2014 and 2023. Fewer than 3.2% passed all six accessibility checks used in the study, including document tagging, heading structure and table headers [2]. Although the study focused on academic publications, it demonstrates that online availability alone does not ensure accessibility.

With the advancement of Artificial Intelligence (AI), with the use of automation, it can reduce some of this work, but extraction errors still require attention. Checking every element manually limits the time saved, while publishing unchecked results can allow mistakes to reach readers. The problem addressed by Konverter is how to reduce manual conversion and review effort while preserving document meaning, supporting accessible publication and keeping people responsible for the final output.

2.3 Motivation and significance

For Komosion and VLRC, the main motivation is to make report preparation more efficient. Konverter uses confidence scores to help reviewers prioritise elements that may need correction. This aims to reduce unnecessary checking and allow staff to spend more time resolving important issues. Reviewers can compare the extracted content with the original PDF and make corrections before approval. Confidence scores provide guidance on which elements to focus on, and the final publishing decision remains with a person.

The project also aims to improve access for readers. A well-structured HTML page can provide clear headings, links between sections and layouts that adapt to different screen sizes. It can also support people who use screen readers. These features can help members of the public, researchers and other readers find relevant information in lengthy legal reports.

Research supports the potential of this approach. SciA11y, a system that converts scientific PDFs into HTML with navigation features for screen-reader users. The researchers generated HTML versions of more than 12 million papers [3]. Their work demonstrates the scale at which document conversion can be applied and supports providing alternative reading formats. For Konverter, the quality and accessibility of the published pages still need to be checked to confirm these benefits and scability.

The project also makes document content easier to reuse. Once information has been extracted, organised and reviewed, other applications can use it without processing the original PDF again. Konverter's document chatbot is one example of this use. Further applications could include searching across reports, finding related information and connecting reviewed content to other systems.

For Komosion, this provides a starting point for supporting other clients with similar publishing needs. The workflow could be adapted to different document collections, although their layouts and content would require further testing. The wider significance of the project is therefore its potential to help organisations turn existing PDFs into information that is easier to access, publish and use in other applications.

3 Literature review & market research

In this section, you will perform some market research and literature review to determine (a) the current and alternative products to what you are working on and (b) the techniques/designs/technologies that are relevant to your solution. For example, if your project includes designing a visualization dashboard, then you may compare it with existing products such as PowerBI, Tableau, Qlik etc. On the other hand, if you are working on a deep learning problem, then your literature review may include the different neural network architects that you may consider for your project.

From doing these market research and literature review, you will be able to justify your project's contributions more clearly and critically. It will also help you to make plans for your project. Besides the practical usefulness of your project, you may want to discuss how your project can contribute to and expand theoretical knowledge as well if applicable.

Update from progress report & completion plan: provide additional market and literature review for any changes or new considerations that you may have for your project. For instance, if your project has expanded its scope and a new function is needed, then you will provide a literature review on the technologies that are relevant to that new function or scope.

Recommended length: 4 to 6 pages

Table 3-1: Example of table and caption presentation

Letter name	Uppercase	Lowercase	Letter name	Uppercase	Lowercase
Alpha	A	α	Nu	N	ν
Beta	B	β	Xi	Ξ	ξ
Gamma	Γ	γ	Omicron	O	o
Delta	Δ	δ	Pi	Π	π
Epsilon	E	ε	Rho	P	ρ
Zeta	Z	ζ	Sigma	Σ	σ

Eta	H	η	Tau	T	τ
Theta	Θ	θ	Upsilon	Υ	υ
Iota	I	ι	Phi	Φ	φ
Kappa	K	κ	Chi	X	χ
Lambda	Λ	λ	Psi	Ψ	ψ
Mu	M	μ	Omega	Ω	ω

4 Solution design

This is the key section of the Progress Report.

In this section, you will first remind the readers about the scope of work and the performance metrics that were discussed in the previous Project Proposal, including any changes that happened during your journey so far.

Also in this section, you will provide a detailed design of the project solution. This design will include descriptions of the solution's functions, users, the technologies that have been considered by the team, to name some examples.

Depending on your specific discipline, you will provide the relevant technical diagrams and visualizations that you have studied in previous courses, in order to present the project specifications by following established professional standards.

If you have tables and complex diagrams that provide detailed specifications and designs, you should consider putting them in the Appendix and referring to these tables and diagrams in the main body of the report.

Recommended length: 6 to 10 pages

4.1 Original Scope

The original project proposed by the industry partner Komosion is called Konverter which is an AI-assisted document intelligence system for Komosion Pty Ltd, with the Victorian Law Reform Commission (VLRC) as the main use case. The project aimed to reduce the manual effort required to convert long and complex legal documents, mainly PDFs, into structured and accessible web content. The existing process was estimated to require approximately four hours of staff effort per document. The proposed goal was therefore to automate much of the document processing while keeping a human reviewer in control of the final output and the whole process should take around 2 hours per document.

The initial scope included document ingestion, structure detection, metadata extraction, confidence scoring, human review, accessible HTML generation, Schema.org JSON-LD generation, audit logging and technical documentation. The proposed processing pipeline combined rule-based techniques, a fine-tuned LayoutLMv3 model and an LLM fallback for ambiguous elements. Each extracted element was intended to receive a confidence score so that uncertain results could be directed to a reviewer rather than requiring the entire document to be checked manually.

4.2 Current Scope

Konverter is structured as a five-stage pipeline: (1) ingestion of the source PDF (2) AI-based layout parsing (3) human review and correction (4) generation of structured, accessible output and (5) publication to the client's website. A document enters the system as an inaccessible PDF and leaves as a semantically structured, accessible web page together with machine-readable data.

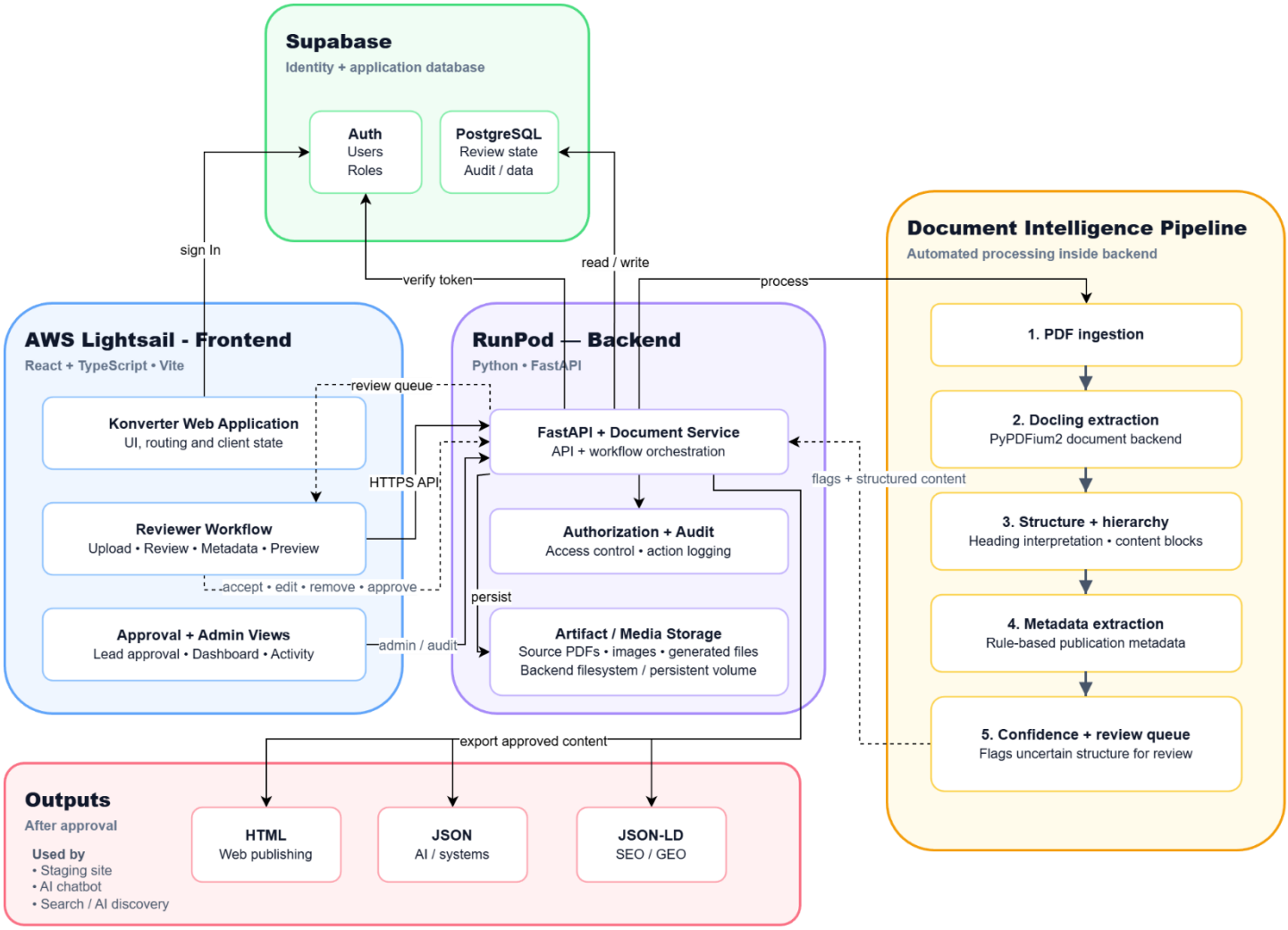


Figure 1. System Architecture

The runtime is divided across three deployable, each chosen to match the resource profile of the work it performs:

Frontend: a React single-page application that provides the reviewer console, served as static assets from AWS LightSail.

Backend API: a FastAPI application that handles authentication, the review workflow, metadata, exports and publishing. It runs continuously on a CPU-only RunPod pod.

Document-parsing worker: the GPU-dependent layout-parsing stage, isolated on a RunPod serverless GPU endpoint that runs only while a document is being processed.

[content: a short description of the AI layout-parsing model and the review-interface design belongs here or in adjacent subsections authored by the model and front-end owners.]

4.3 Split compute architecture and rationale

The central infrastructure design decision was to separate the compute along its resource profile. The document-parsing stage uses a deep-learning layout model (Docling) that benefits substantially from a GPU, whereas the remainder of the system: serving the dashboard, managing the review workflow and generating exports requires only modest CPU resources. Running a single GPU-equipped server continuously to satisfy the occasional parsing workload would mean paying for an expensive accelerator that sits idle for most of the time.

The adopted design therefore runs the always-on backend on an inexpensive CPU pod and delegates parsing to a serverless GPU worker that scales to zero when idle and is billed only for the seconds it is actively processing a document. When the backend is configured for remote parsing, it uploads the source PDF to shared object storage, invokes the serverless endpoint, polls the job until completion, and then retrieves the parsed result and continues the CPU-bound stages. The two services communicate through a Supabase storage bucket rather than a shared disk, which keeps them independent.

This design delivers three benefits beyond cost. **Cost efficiency:** the always-on hosting baseline is reduced from approximately **US$320** per month for a continuously running GPU server to approximately **US$104** per month for the CPU pod, with GPU time billed per document (detailed in Result Analyses). **Fault isolation:** a failure while parsing a large document cannot affect the dashboard, because the two run as separate services. **Independent scalability:** multiple documents can be parsed in parallel on separate serverless workers without the API competing for the same machine.

The principal trade-off is cold-start latency: the first document processed after an idle period must wait for a GPU worker to initialise. For a review-oriented workflow, in which documents are submitted and then reviewed over a period of minutes, this latency is an acceptable cost, it is quantified in the Result Analyses section

4.4 Technology stack and required resources

The technologies were selected to match the project's constraints of low cost, accessibility of output, and a small team operating without dedicated operations staff. Table 4.2 summarises the stack and the rationale for each choice.

Layer / component	Technology	Rationale
Frontend	React, TypeScript, Vite	Component-based reviewer console, static build hosts cheaply on Lightsail.
Backend API	Python, FastAPI	Asynchronous API suited to polling long-running parsing jobs, strong typing via Pydantic.
Document parsing	Docling layout model	Open-source layout parser producing structured document elements, runs on GPU for speed.
Database & auth	Supabase (PostgreSQL)	Managed Postgres with built-in authentication; hosts the documents, audit and publication records.
Object storage	Supabase Storage	Shared bucket for passing PDFs and parsed results between the backend and the serverless worker.

Backend hosting	RunPod (CPU pod + serverless GPU)	Serverless GPU enables scale-to-zero billing, CPU pod hosts the always-on API.
Frontend hosting	AWS Lightsail	Fixed-price, simple hosting alongside the client's existing infrastructure.
Containerisation	Docker	Reproducible images for both the backend and the GPU worker.
AI chat	OpenAI API	Document-grounded question answering over an approved report.

Table 4-2: Technology stack and selection rationale.

4.5 Data storage and persistence

Persistence is provided by Supabase in two forms. Structured records the document register and the audit trail are stored in PostgreSQL, while the source PDFs and generated artefacts are stored in a Supabase storage bucket that also serves as the exchange medium between the backend and the serverless parsing worker.

A design consideration that received significant attention was making the backend pod stateless, so that it could be stopped, replaced or relocated without data loss. A migration to make Supabase the single source of truth for all document artefacts was designed and partially implemented. At the time of writing this migration is not fully complete**:** the backend persists document artefacts on a RunPod network volume attached at the data directory, which preserves data across pod restarts and redeployments within the same region. The audit and publication records already reside in Supabase and are independent of the pod. Completing the stateless migration is identified as future work.

4.6 Security and audit design

Because Konverter processes documents for a public legal body, the integrity and provenance of the review process are important. The design incorporates three security elements.

Authentication and access control. User authentication is delegated to Supabase, the backend validates the resulting tokens on every request, and administrative actions are restricted accordingly.

Tamper-evident audit trail. Every consequential action: upload, processing, review edits, approval and publication is recorded in an append-only audit log. Each entry is cryptographically hash-chained to the previous entry, so that any later modification or deletion of a record breaks the chain and becomes detectable. The log is additionally configured at the database level to reject updates and deletions, making it insert-only. This provides a verifiable record of who did what and when, which is appropriate for legal-publishing provenance.

Secrets management, informed by a real incident. During the delivery phase an API key used by the chatbot exhibited abnormal usage and was treated as compromised. The response led as part of this infrastructure work was to rotate the affected key, identify the likely exposure vectors, move all secrets into the platform's encrypted secret storage, and change the backend container image from public to private visibility on the registry. This incident directly shaped the standing secrets-handling procedure documented in the operational handover and demonstrates the practical security considerations of deploying an AI service.

4.7 Publishing integration and duplicate safeguard

The final stage publishes an approved document directly into the client's WordPress site as a correctly formatted draft, removing the manual copy-and-paste step that the client previously relied on. A design constraint of the client's publishing API is that it can only create pages, it has no facility to update an existing page. Publishing the same document twice would therefore create duplicate pages.

To prevent silent duplication, every successful publication is recorded durably in the database. Before publishing, the backend checks this record, if the document was already published and its content has since changed, the system does not silently create a second page but instead returns a warning that names the existing page and requires the reviewer to confirm explicitly before an additional page is created. This converts an easy, invisible error into a deliberate, informed choice.

4.8 Deployment and operational resources

Both the backend and the parsing worker are packaged as Docker images and deployed to RunPod, the frontend is built and served from AWS Lightsail. Configuration is supplied entirely through environment variables, with secrets held in the platform's encrypted secret store. Deployment is currently performed manually: images are built, pushed and deployed, and database schema changes are applied by hand, and the absence of an automated build-and-deploy pipeline is noted as a future improvement. To ensure the system can be operated after the team's involvement ends, an operational handover document was produced covering configuration, the build-and-deploy sequence, the database schema scripts, restart and recovery procedures, and known limitations.

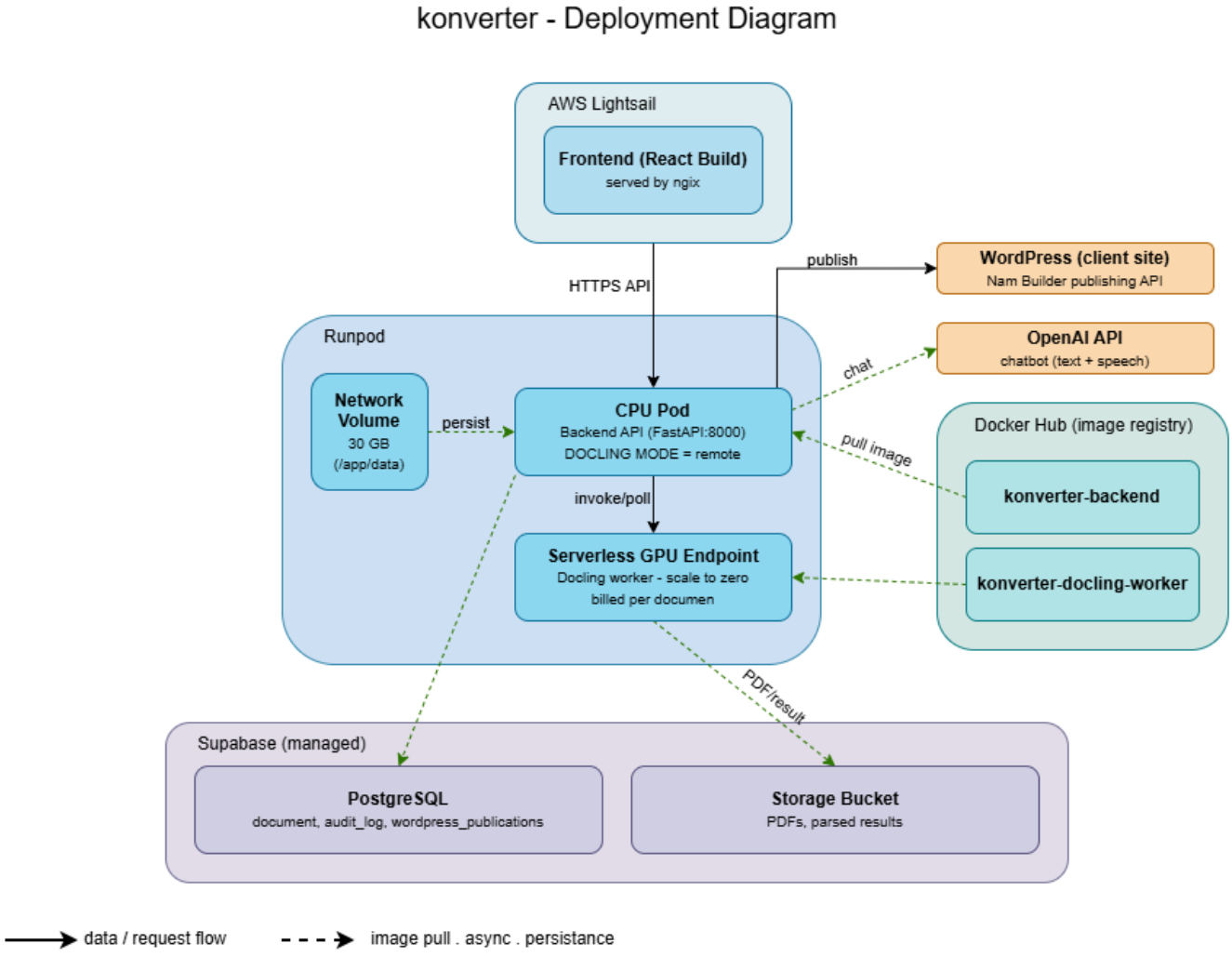


Figure 4-6: Deployment Diagram

5 Result analyses & discussions

Present key results and accomplishments of your capstone projects. Do you accomplish all key deliverables proposed in the project proposal and meet or exceed all projects requirements?

Analyze and discuss the results and findings, performance and key characteristics of the final product, potential issues for future work and consideration

Recommended length: 4 to 6 pages

5.1 Deliverables Achieved (mapped to the Project Completion Plan milestones)

Against the deliverables set out in the project proposal and completion plan, the infrastructure, deployment and security objectives were met. Table 5.1 maps each objective to its outcome

Milestone	Key deliverable(s)	Target (SMART metric, condensed)	Status	Result / evidence
M1: Foundation & Ingestion	Ingestion & pre-processing layer; rule-based metadata extraction	≥95% PDFs processed; OCR ≥98% char accuracy; 5 metadata fields correct for ≥90% docs; ≤300 s preprocessing for 250-page PDF	**Achieved***	Ingestion pipeline (Docling) and 5-field metadata extraction implemented and running in production. *OCR/character-accuracy and metadata-accuracy percentages to be confirmed by the model/data owner from the DocLayNet/client-corpus evaluation.
M2: AI Document Intelligence	Custom LayoutLMv3 model; heading-hierarchy reconstruction	LayoutLMv3 ≥85% F1; hierarchy ≥90% accuracy; ≥95% low-confidence predictions flagged	**Achieved***	Completion plan reports fine-tuned LayoutLMv3 at F1 ≈ 0.97 (exceeds ≥85% target). *Confirm the final production parser (LayoutLMv3 vs Docling) and the hierarchy/flagging figures with the

				model owner; note any pipeline change in the text.
M3: Backend Integration	Inference Backend API	API ≥95% successful requests; ≤5 min end-to-end for 250-page PDF; Docker runs on all planned environments	**Achieved (target revised)**	FastAPI backend built, Dockerised, and deployed (RunPod CPU pod + serverless GPU worker; also runnable locally).
M4: Human-in-the-Loop	Human Review Interface; Immutable Audit Trail	Reviewers resolve flagged items ≤10 min/doc; ≥95% errors detected; 100% of actions recorded with full traceability	**Achieved***	Immutable audit trail delivered: append-only, hash-chained, insert-only log capturing uploads, edits, approvals and publication with full traceability (100% target met). Review interface delivered. *Usability metrics (≤10 min; ≥95% error detection) to be confirmed by the interface owner via the timed evaluation.
M5: Output & Documentation	Publishable Output; Technical Document	100% exported to accessible HTML + Schema.org JSON-LD; HTML passes WCAG 2.1; JSON-LD validates; complete technical documentation	**Achieved***	System exports accessible HTML and Schema.org JSON-LD, and additionally publishes directly to the client's WordPress site (beyond original scope). Operational/handover technical documentation produced. *WCAG 2.1 and Schema.org validation passes to be confirmed by the output owner using the validation tools.
M6: Future Enhancement Planning	RAG Roadmap (design only)	Complete architecture proposal (diagrams, data flow, integration, roadmap); reviewed & approved by supervisors	**Confirm with owner**	Design-only deliverable. Confirm with the owner whether the RAG roadmap document has been produced and supervisor-approved; update status accordingly.

Table 5-1: Infrastructure deliverables and outcomes.

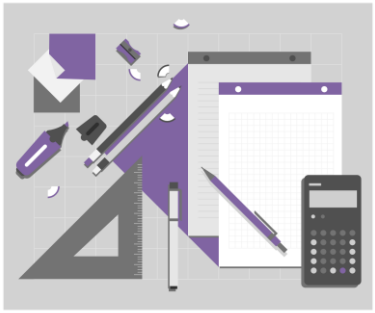


Figure 5-2: A Gantt chart with brief discussion on your project timeline will work well

6 Sumary and reflection

In this section, you will reflect on the capstone project journey so far to identify what worked and what did not work, and discuss the reasons and lessons learned that you have drawn from working on the project. At the end of your reflection, you will discuss the key considerations to ensure success in the next phases of your project, and provide some recommendations.

Please note that the reflection should be objective and constructive, and written from the perspective of the whole team as a whole rather than reflecting individual's viewpoints. This section should also not be used to name and criticize individuals for their mistakes, or to blame people or situations for undesirable events that took place in your journey. You are encouraged to discuss as a team to decide the key points to be included in this section first before starting to write it.

Recommended length: 1 page

7 References

[1] C. Auer *et al.*, *Docling Technical Report*. 2024.

[2] L. Wang *et al.*, *Improving the Accessibility of Scientific Documents: Current State, User Needs, and a System Solution to Enhance Scientific PDF Accessibility for Blind and Low Vision Users*. 2021.

[3] A. Kumar and L. L. Wang, "Uncovering the New Accessibility Crisis in Scholarly PDFs: Publishing Model and Platform Changes Contribute to Declining Scholarly Document Accessibility in the Last Decade," presented at the Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility, St. John's, NL, Canada, 2024. [Online]. Available: https://doi.org/10.1145/3663548.3675634.

8 Appendices

Additional information & relevant data for your proposal can be presented in this section.

9 Meeting journals

Please include all minute minutes of the project in this section.